Manal Anur
Python_Lab
Link https://github.com/manalan294-creator/PYTHON_LAB

YTHON PROJECT SETUP, GIT, AND GITHUB WORKFLOW

Part A: Project Setup Using the CLI

1. Commands Used

```
cd ~
mkdir python_lab
cd python_lab
mkdir src tests docs
touch src/main.py src/utils.py src/config.py
echo "My Python Lab Project" > docs/README.md
ls -R
```

2. Explanation of the Commands

The `cd ~` command moves to the home directory. The `mkdir python_lab` command creates a new project directory, and `cd python_lab` moves into it. The `mkdir src tests docs` command creates three subdirectories. The `touch` command creates three empty Python files inside the `src` folder. The `echo` command, together with the `>` redirection symbol, writes the text “My Python Lab Project” into `docs/README.md`. Finally, `ls -R` displays the folders and files recursively.

3. Expected Directory Structure

```
python_lab/
├── src/
│   ├── main.py
│   ├── utils.py
│   └── config.py
├── tests/
└── docs/
    └── README.md
```

4. Why Separating Folders Is Good Practice

Separating a project into `src`, `tests`, and `docs` makes it organized and easier to maintain. The `src` folder contains the program's source code, the `tests` folder is used for testing the code,

and the `docs` folder contains documentation. This structure helps developers find files easily, identify errors, and work together more efficiently, even on small projects.

5. Conclusion

In conclusion, I created the `python_lab` project using command-line instructions. I created the required folders and files and displayed the directory structure using `ls -R`. This exercise helped me understand basic command-line operations and the importance of organizing a Python project.